

Student ID: 1123301

Student Name: 黄立习

Course: Data Structures (CSE CS203A)

Assignment III: Linked List Selection Sort

Student Worksheet Companion

A1. Linked List Representation Drawing (5 pts)

- a. (2 pts) Instructions: Draw a visual representation of a single node with next pointer that contains the initialized integer 10

[10 | •] → NULL

- b. (3 pts) Linked list representation with the given integers (Hint: For safety and clarity, include identifiable head and tail nodes)

Example: the input integers are (10, 20) and linked list representation will be [10 | •] → [20 | •] →

[10 | •] → [20 | •] → NULL

A2. Populate with Integers (32 pts; 2 pts for each)

Fill the given integers (60, 24, 15, 42, 20, 11, 90, 8) into the above structures.

Annotate:

Node #	Value	Next Pointer
1	60	→ Node 2
2	24	→ Node 3
3	15	→ Node 4
4	42	→ Node 5
5	20	→ Node 6
6	11	→ Node 7
7	90	→ Node 8

Student ID: 1123301

Student Name: 蔡文翔

8

[8]

→ [NULL]

A3. Selection Sort – First Three Steps (45 pts; 15 pts for each step)

Step Trace Table (Linked list):

Step 1 is the example to help you to complete step 2 to 4.

Step 1 (i = head = 60): Traverse list to find minimum value 8 → call swap function Yes; swap (60, 8).

head → [8 | •] → [24 | •] → [15 | •] → [42 | •] → [20 | •] → [11 | •] → [90 | •] → [60 | NULL]

Step 2 (i = 24): Minimum value [11] → call swap function Yes / No; swap ([24], [11]).

head → [8 | •] → [11 | •] → [15 | •] → [42 | •] → [20 | •] → [24 | •] → [90 | •] → [60 | NULL]

Step 3 (i = 15): Minimum value [15] → call swap function Yes / No; swap ([], []).

head → [8 | •] → [11 | •] → [15 | •] → [42 | •] → [20 | •] → [24 | •] → [90 | •] → [60 | NULL]

Step 4 (i = 42): Minimum value [20] → call swap function Yes / No; swap ([42], [20]).

head → [8 | •] → [11 | •] → [15 | •] → [20 | •] → [42 | •] → [24 | •] → [90 | •] → [60 | NULL]

Student ID: 1123301

Student Name: 黃劍庭

A4. Discussion (68 pts)

Guiding Questions:

- How many swaps/exchanges are performed? 最壞情況是 $n-1$ 次 (每次最小值都剛好不在位置)
- How expensive is traversal for arrays vs. linked lists? array: index 存取 $O(1)$ 掃描 $O(n)$, list: 皆 $O(n)$
- What memory/overhead differences do you see? array 要連續記憶體, link list 多存指標開銷, 但非連續
- Which representation is easier to visualize? 陣列好看易懂
- Which would you choose for implementing selection sort and why? No, 處理麻煩且無特別優勢

Time complexity comparison (14 pts, 1pt for each)

Aspect / Operation	Array	Linked List	Explanation
Access Element	(1)	(2)	Array allows direct indexing; linked list needs traversal.
Find Minimum	(3)	(4)	Both must scan all remaining elements/nodes.
Swap Operation	(5)	(6)	In array, swap by indices; in linked list, swap node values.
Traversal Between Elements	(7)	(8)	Linked list traversal requires pointer navigation.
Overall Time Complexity (Selection Sort)	(9)	(10)	Both involve nested traversal to find minima; linked list adds traversal overhead.
Space Complexity	(11)	(12)	Both sorts are in-place if swapping values, not nodes.
Implementation Overhead	(13) Low or Moderate	(14) Low or Moderate	Linked list needs pointer operations and careful null checks.

Student ID: 1123301

Student Name: 許立偉

(1)	$O(1)$	(2)	$O(n)$
(3)	$O(n)$	(4)	$O(n)$
(5)	$O(1)$ by index	(6)	$O(1)$ by value
(7)	$O(n)$	(8)	$O(n)$
(9)	$O(n^2)$	(10)	$O(n^2)$
(11)	$O(1)$	(12)	$O(1)$
(13)	low	(14)	Moderate

Student ID: 1123301

Student Name: 李羽峰

Characteristics (54 pts, 3 pts for each)

Aspect	Array	Linked List
Storage	(1)	(2)
Access	(3)	(4)
Extra Variables	(5)	(6)
Traversal	(7)	(8)
Overhead	(9)	(10)
Visualization	(11)	(12)
Swaps	(13)	(14)
Flexibility	(15)	(16)
Overall	(17)	(18)

(1) Stored in contiguous memory blocks, size is fixed after declaration.

(2) Stored in non-contiguous memory, each node dynamically allocated.

(3) Direct access using index ($O(1)$)

(4) Sequential access only, must traverse from head ($O(n)$)

(5) Loop counters only

(6) Only i and j are enough, because swap by val, do not need prev informations, need danny.

(7) Traversal by index increment

(8) Traversal by pointer navigation between nodes

(9) Lower overhead

(10) Higher due to pointers and dynamic memory

(11) Clear, simple boxes and indices

(12) Nodes with arrow, more complex but intuitive for links

(13) By index, ~~stable~~ sorting algorithm

(14) By value, ~~unstable~~ sorting algorithm

(15) Fixed length

Student ID: 1123301

Student Name:

(16) Dynamic length, ^{flexible}
more

(17) Faster and simpler for sorting.

(18) More flexible but ^{more} complex and slow for sorting

61